Lab 1

61C Summer 2026

TA intro

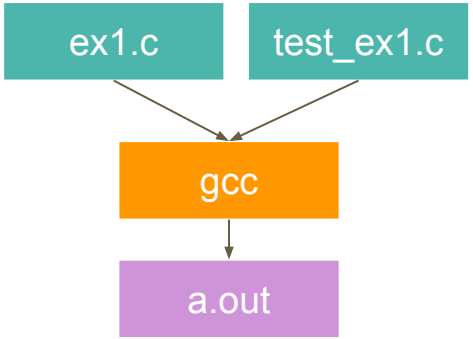
Deadline

This lab's deadline is Tuesday, June 30th at 11:59 PM

Generally, for all labs this semester, one will be due on Tuesday and one will be due on Thursday every week

Compiling a C Program

- `gcc` is used to compile C programs
- `gcc ex1.c test_ex1.c`



- Can specify the name of the executable file with the `-o` flag
  - `gcc -o ex1 ex1.c test_ex1.c`

Running a C Program

- To run an executable located in the current directory, use
  `./<executable_name>`
  - `./ex1`
- The dot refers to the current directory
- If you want to run a file in a different directory, specify the path after the dot
  - `./path/to/file/ex1`

Variable Types and Sizes

- `char` = 1 byte (8 bits)
- `short` = usually 2 bytes (16 bits)
- `int` = usually 4 bytes (32 bits)
  - `unsigned int`
- `float` = 4 bytes (32 bits)
- `double` = 8 bytes (64 bits)
- `long` = at least 4 bytes (32 bits), can be longer
- `long long` = at least 8 bytes (64 bits), can be longer

Guarantee: sizeof(long long) >= sizeof(long) >= sizeof(int) >= sizeof(short)

To know for sure what size your variable is, **use `uintN_t` or `intN_t` types.**

Defining a Function

Specify return type, function name, and function parameters.

```
int add(int x, int y) {      return x + y;     }

      void nothing() {      return;     }
```

Conditionals

If-else

```
if (condition) {
    do this;
} else if (condition) {
    do this;
} else {
    do this;
}
```

Switch statements

```
switch (expression) {
    case constant1:
        do these;
        break;
    case constant2:
        do these;
        break;
    default:
        do these;
}
```

Loops

While loop

```
while (condition) {
    do this;
}
```

For loop

```
for (int i = 0; i < 10; i++) {
    do this;
}
```

Pointers

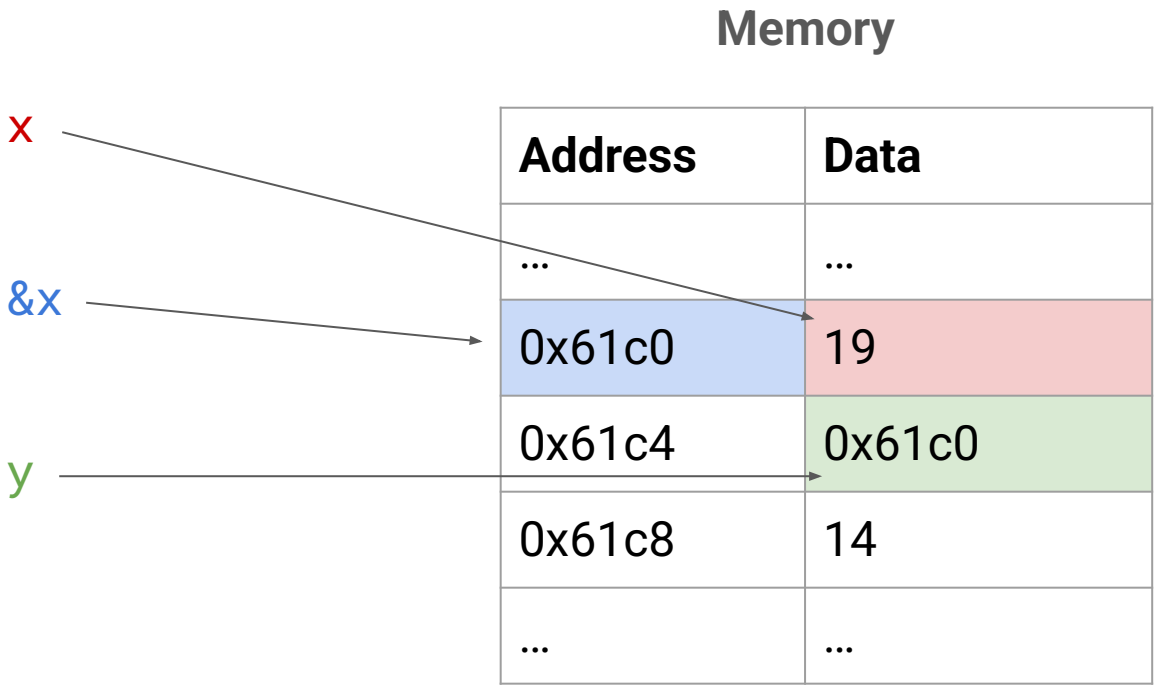
```
int* x_ptr;
int x = 100;
x_ptr = &x;
```

- *
  - Type definition
    - e.g. int* x_ptr;
    - Add an * after a type makes the type a pointer type. In the example, we declare x_ptr variable as integer pointer type, and assign the address of variable x as its value.
    - You can also do int**, int***, int****, ...
  - Dereference
    - e.g. *x_ptr
    - Add an * before a variable gets the value at address [variable]
- &
  - Get the address of a variable (opposite of dereference)
    - &x

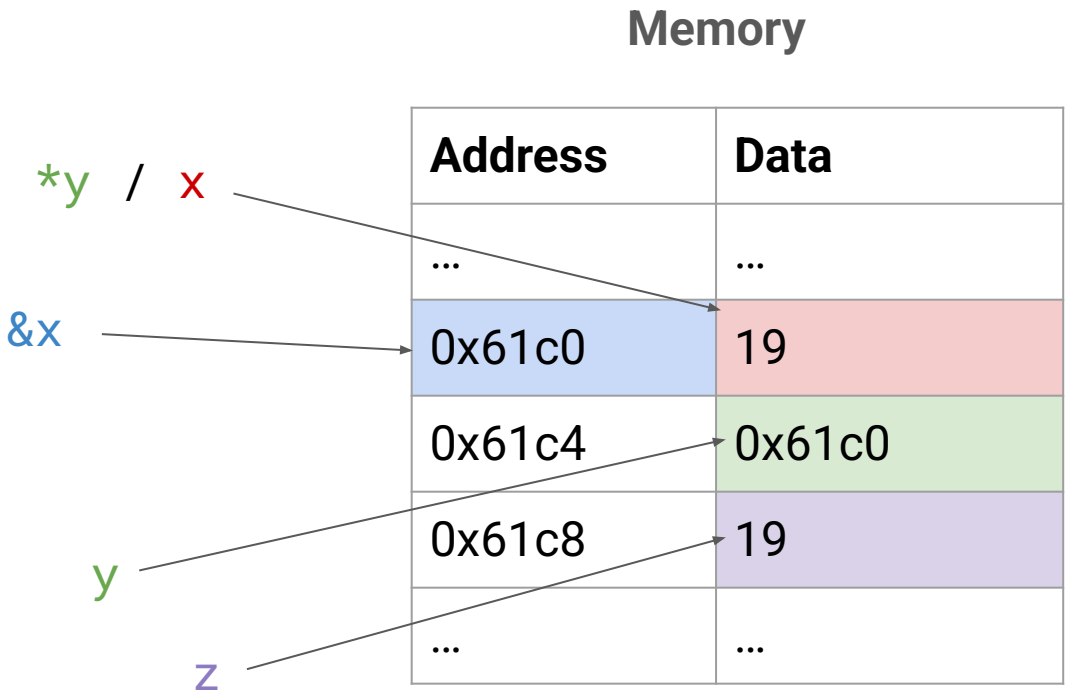
Assigning

```
int x = 19;
int* y = &x;
```

might produce something like:



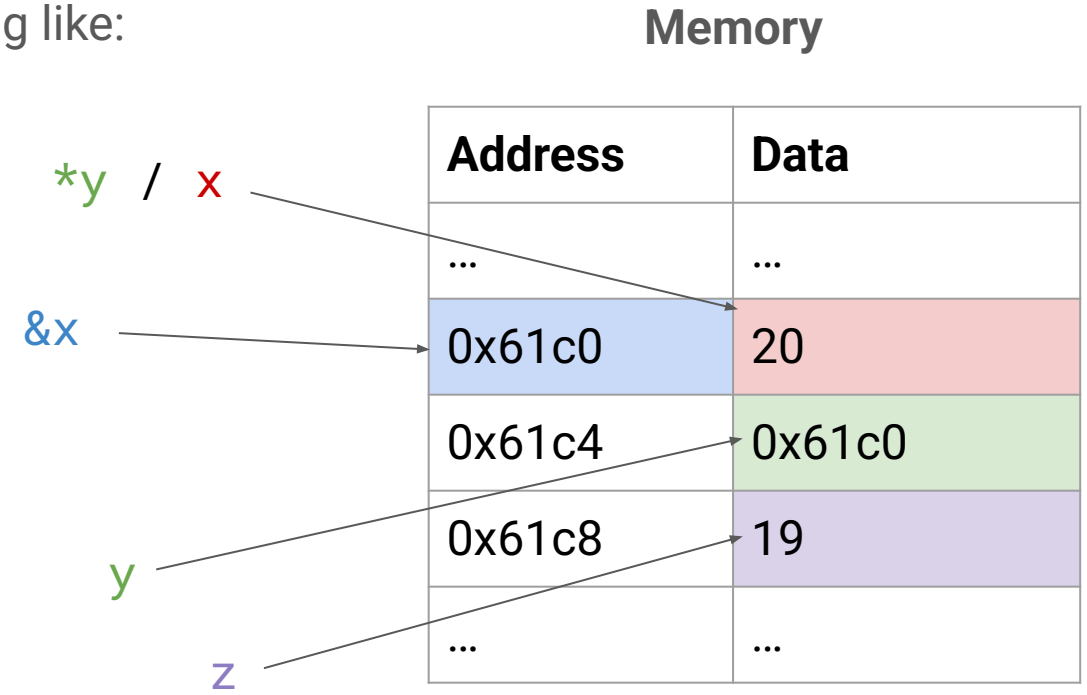
Assigning
`(int* y = &x;)`
`int z = *y;`
might produce something like:



Assigning

```
(int* y = &x;)
int z = *y;
*y = 20;
```

might produce something like:



Name	Value
x	19
&x	0x61c0
y	0x61c0
*y	19

Address	Binding	Data
0x61c0	x	19
0x61c4	y	0x61c0
0x61c8	z	14

&x gets the address of the variable x

Name	Value
x	19
&x	0x61c0
y	0x61c0
*y	19

Address	Binding	Data
0x61c0	x	19
0x61c4	y	0x61c0
0x61c8	z	14

To create a pointer, we need to declare y to be a pointer type:

```
int* y = &x;
```

Name	Value
x	19
&x	0x61c0
y	0x61c0
*y	19

Address	Binding	Data
0x61c0	x	19
0x61c4	y	0x61c0
0x61c8	z	14

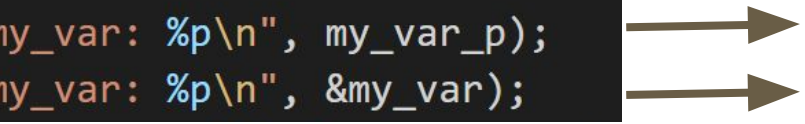
*y follows the pointer in y, and gets the data stored at that address

Name	Value
x	19
&x	0x61c0
y	0x61c0
*y	19

Address	Binding	Data
0x61c0	x	19
0x61c4	y	0x61c0
0x61c8	z	14

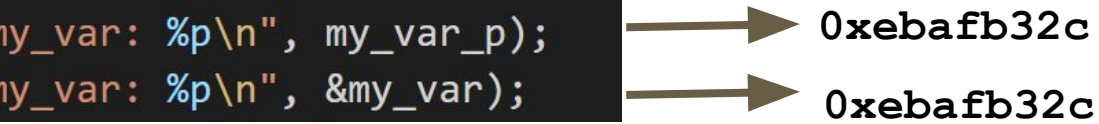
Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
```



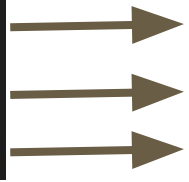
Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
```



Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
```



0xebafb32c

0xebafb32c

What will this print?

Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
```

0xebafb32c

0xebafb32c

Another address, ex:

0xebafb320

Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
    *my_var_p += 2;
    printf("my_var: %d\n", my_var);
```

0xebafb32c

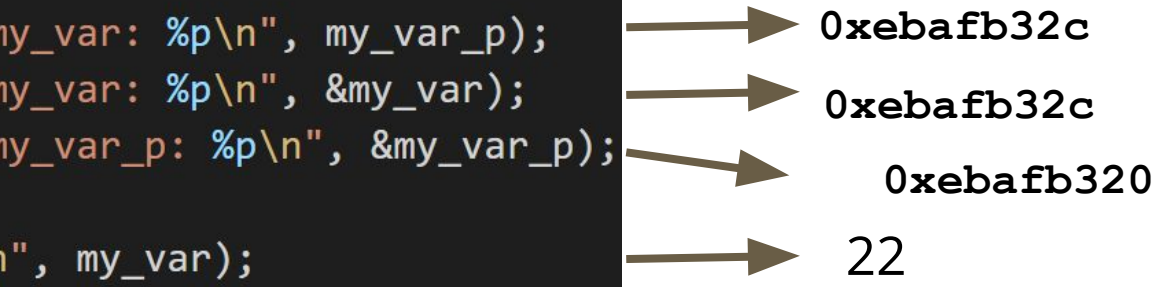
0xebafb32c

0xebafb320

What will this print?

Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
    *my_var_p += 2;
    printf("my_var: %d\n", my_var);
```



Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
    *my_var_p += 2;
    printf("my_var: %d\n", my_var);
    printf("my_var: %d\n", *my_var_p);
    return 0;
}
```

`0xebafb32c`

`0xebafb32c`

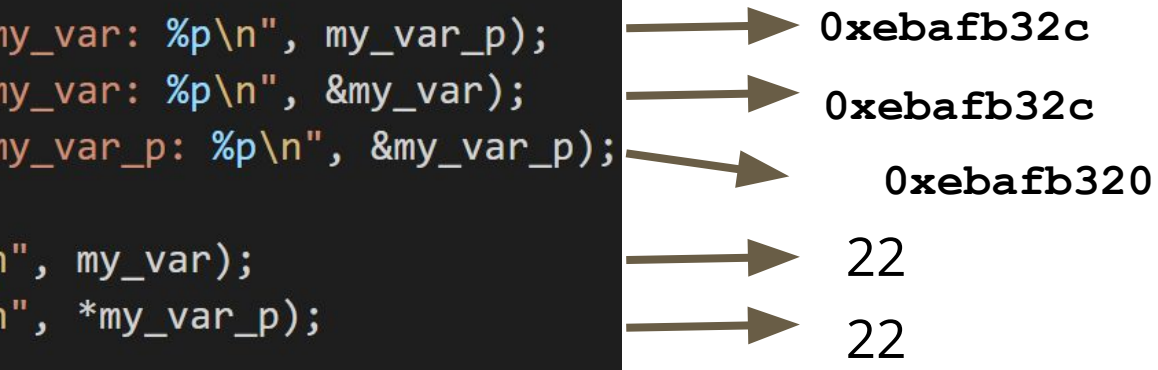
`0xebafb320`

22

What will this print?

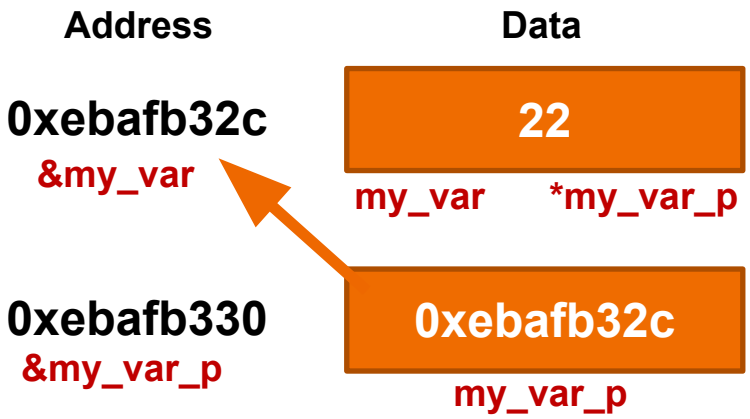
Pointers

```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
    *my_var_p += 2;
    printf("my_var: %d\n", my_var);
    printf("my_var: %d\n", *my_var_p);
    return 0;
}
```



Pointers

&x = address of x
*x = contents at x




```c
#include <stdio.h>

int main () {
    int my_var = 20;
    int* my_var_p;
    my_var_p = &my_var;
    printf("Address of my_var: %p\n", my_var_p);
    printf("Address of my_var: %p\n", &my_var);
    printf("Address of my_var_p: %p\n", &my_var_p);
    *my_var_p += 2;
    printf("my_var: %d\n", my_var);
    printf("my_var: %d\n", *my_var_p);
    return 0;
}
```

```
Address of my_var: 0xebafb32c
Address of my_var: 0xebafb32c
Address of my_var_p: 0xebafb330
my_var: 22
my_var: 22
```

Arrays

- A block of memory: size is **static**
  - `int arr[2];`
  - `int arr[] = {1, 2};`
- Accessing elements: array indexing
  - `arr[1]`
- An array variable is a “pointer” to the first element.
  - You can use pointers to access arrays!
  - `arr[0]` is the same as `*arr`
  - Can use pointer arithmetic to move the pointer
    - Each operation automatically moves the size of one whole “type” that ptr points to
    - `arr[1]` is the same as `*(arr + 1)`
- Arrays do not keep track of their own size

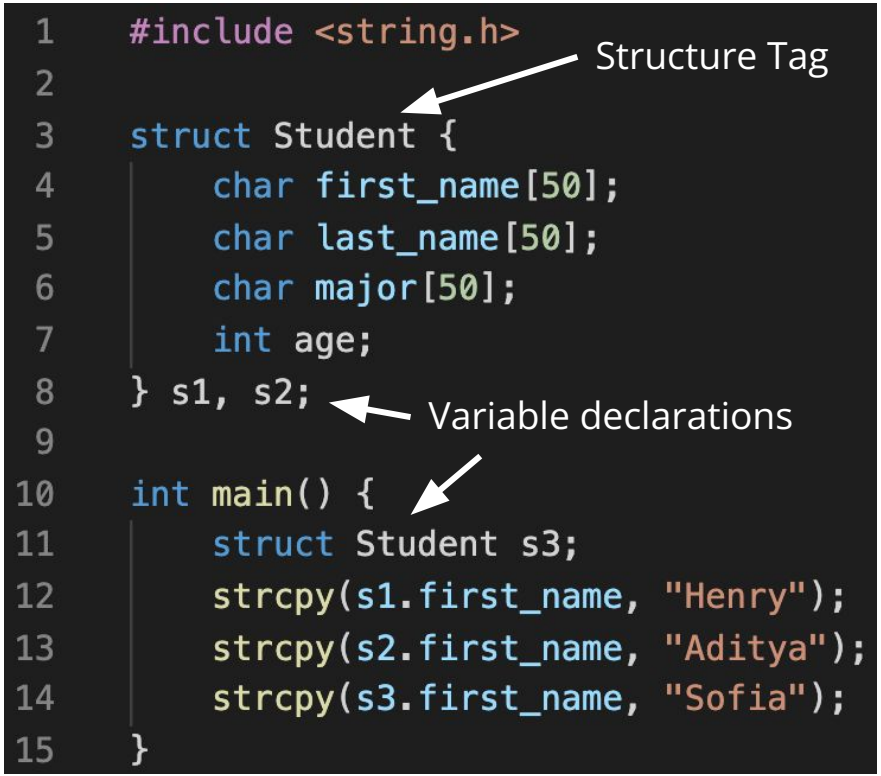
Strings

- An array of **chars** representing individual characters
  - Ends in a null terminator '\0'
- strlen(char* string)
  - returns the length of the string (num chars until the null terminator)
- strcpy(char* dest, char* src)
  - copies the string stored in src to the location dest, until and including the null terminator
  - strcpy does **not** check that you have allocated enough space at dest for the src string!
    - there is a safer version of strcpy, strncpy (documentation)

```
char str1[] = "hello";
char str2[6]; //dont forget to add space for the null terminator!
strcpy(str2, str1);
printf ("stri: %s\nstr2: %s\n", str1, str2); //str1: hello
                                             //str2: hello
return strlen(str2); //returns 5, not 6
```

Structs

- What do they allow us to do?
- What is a structure tag?
- What are the two ways to declare struct variables?
- How do we access the members of a struct?



Structs

- What do they allow us to do?
- What is a structure tag?
- What are the two ways to declare struct variables?
- How do we access the members of a struct?

```c
#include <string.h>

struct Student {
    char first_name[50];
    char last_name[50];
    char major[50];
    int age;
} s1, s2;

int main() {
    struct Student s3;
    strcpy(s1.first_name, "Henry");
    strcpy(s2.first_name, "Aditya");
    strcpy(s3.first_name, "Sofia");
}
```

Pointers to Structs

- Pass a pointer of a struct to a function so that you can edit the contents
- Access the struct contents by:
  - (*student).major
  - student->major

```
major: chemistry
major: biology
```

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char first_name[50];
    char last_name[50];
    char major[50];
    int age;
} Student;

void update_major (Student *student, char *new_major) {
    //strcpy((*student).major, new_major);
    strcpy(student->major, new_major);
}

int main() {
    Student s1;
    strcpy(s1.major, "chemistry");
    printf("major: %s\n", s1.major);
    update_major(&s1, "biology");
    printf("major: %s\n", s1.major);
}
```

Arrow operator

Structs

- typedef
  - Lets you avoid rewriting `struct` every time you want to declare a new struct variable
  - Can no longer declare variables in the struct definition

```c
#include <string.h>

typedef struct {
    char first_name[50];
    char last_name[50];
    char major[50];
    int age;
} Student;

int main() {
    Student s1, s2, s3;
    strcpy(s1.first_name, "Henry");
    strcpy(s2.first_name, "Aditya");
    strcpy(s3.first_name, "Sofia");
}
```